

Architectural Reflection: Lab 2 — Bridge Pattern

Course: Integrative Programming and Technologies (IPT)

Module: Structural Design Patterns in Web Application Development

Topic: Times Formatter × Data Source × Compression (Handle-Body Decoupling)

1. Spotting the Smell: Cartesian Class Explosion

In `without_web.php`, the reporting subsystem attempts to manage multiple orthogonal concerns—formatting, data fetching, and compression—using class inheritance:

AttendanceJsonGzipReport, AttendanceJsonPlainReport, AttendanceTextGzipReport ...

This approach exhibits disastrous combinatorial scaling:

1. **Combinatorial Explosion ($M \times N \times K$):** With 2 formats, 2 data sources, and 2 compression modes, inheritance demands $2 \times 2 \times 2 = 8$ classes. Adding a single new format (e.g., CSV) requires 4 additional subclasses; adding ZIP compression requires 6 more.
 2. **Duplication of Implementor Logic:** The identical network fetch (`file_get_contents(...)`) and compression logic (`gzencode(...)`) are redundantly copy-pasted across dozens of leaf classes.
 3. **Inability to Reconfigure at Runtime:** An instantiated report cannot change its compression strategy or data origin dynamically without re-instantiating an entirely different subclass.
-

2. Architectural Fix: The GoF Bridge Pattern

The refactored design (`with_pattern.php`) adopts the **Bridge Pattern**, partitioning the problem space into an **Abstraction hierarchy** and two independent **Implementor hierarchies**:

- **Abstraction:** `TimesFormatter` defines the high-level report generation workflow (`generate()`) and delegates raw record transformation to subclasses via `toText(array $records): string`.
- **Implementor A (DataSource):** Encapsulates where records originate (`ApiDataSource` for live HTTP JSON; `FileDataSource` for local fallback).

- **Implementor B (Compressor):** Encapsulates output compression (`GzipCompressor` vs. `NoneCompressor`).

By composing the implementors into the abstraction via aggregation (`TimesFormatter o--> DataSource` and `TimesFormatter o--> Compressor`), the class count collapses from multiplicative ($M \times N$) to additive ($M + N + K$). Furthermore, adding `setCompressor(Compressor $c)` allows runtime strategy mutation.

3. Academic Grounding (Gamma et al., 1994)

"Decouple an abstraction from its implementation so that the two can vary independently."

— Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides (*Design Patterns: Elements of Reusable Object-Oriented Software*, 1994, p. 151)

Buschmann et al. (1996) describe this as the **Handle/Body Pattern**: the "Handle" (client-facing report abstraction) holds a reference to the "Body" (platform-specific implementors). This eliminates permanent compile-time binding between abstractions and platform implementations.

4. Extensibility Demonstration & Practical Web Applications

To prove extensibility, a new format—`TimesheetCsvFormatter`—was added with just 1 **class**. Instantly, without modifying any `DataSource` or `Compressor` class, 4 new functional combinations were unlocked ($3 \times 2 \times 2 = 12$ total combinations powered by only 7 classes total). Runtime compression switching was also proven, shrinking JSON payloads by over 50%.

Real-World Web Applications in PHP Frameworks

- **Database Query Builders (Laravel Eloquent / Doctrine DBAL):** The query builder abstraction (`DB::table(...)`) remains identical while delegating to concrete driver implementors (`MySQLGrammar`, `PostgresGrammar`, `SQLiteGrammar`).
- **Mailing Pipelines (Laravel Mail / Symfony Mailer):** The `Mailable` abstraction encapsulates message content and layout, while bridging to swappable transport implementors (SMTP, Amazon SES, Mailgun, Postmark).
- **Logging (PSR-3 / Monolog):** The `Logger` abstraction delegates to diverse `Handler` implementors (`StreamHandler`, `SyslogHandler`, `SlackHandler`).